

JavaScript Basics

Comprehensive Reference Notes

1. DOM Dynamics

Moving beyond static HTML by manipulating the DOM dynamically using JavaScript.

createElement & appendChild

Use createElement to build new elements, then inject them into the page with appendChild or insertBefore.

```
// Create and append a new element
const li = document.createElement('li');
li.textContent = 'New Item';
li.classList.add('item');

const ul = document.querySelector('#myList');
ul.appendChild(li);           // add at end
ul.prepend(li);              // add at start
ul.insertBefore(li, ul.firstChild); // insert before first child
```

Event Delegation with addEventListener

Instead of attaching listeners to every child element, attach one listener to a parent and use event.target to determine what was clicked. This is more efficient and works for dynamically added elements.

```
// ✗ Inefficient: listener on each button
buttons.forEach(btn => btn.addEventListener('click', handler));

// ☑ Event Delegation: one listener on the parent
document.querySelector('#list').addEventListener('click', (e) => {
  if (e.target.matches('.delete-btn')) {
    e.target.closest('li').remove();
  }
});
```

💡 Event delegation relies on event bubbling — events fired on a child propagate up to ancestor elements.

2. Collection Logic

JavaScript provides powerful tools for iterating and transforming data.

for...in vs for...of

```
const arr = ['a', 'b', 'c'];
const obj = { x: 1, y: 2, z: 3 };

// for...in → iterates over KEYS (use for objects)
for (const key in obj) {
  console.log(key, obj[key]); // 'x' 1, 'y' 2, 'z' 3
}

// for...of → iterates over VALUES (use for iterables)
for (const val of arr) {
  console.log(val); // 'a', 'b', 'c'
```

```
}
```

🔗 Avoid `for...in` on arrays — it can iterate over prototype properties. Use `for...of` or array methods instead.

Functional Array Methods

These methods return new arrays/values without mutating the original — a cornerstone of functional programming in JS.

```
const nums = [1, 2, 3, 4, 5];

// .map() → transform each element, returns new array
const doubled = nums.map(n => n * 2); // [2,4,6,8,10]

// .filter() → keep elements matching condition
const evens = nums.filter(n => n % 2 === 0); // [2,4]

// .reduce() → accumulate to a single value
const sum = nums.reduce((acc, n) => acc + n, 0); // 15

// Chaining methods
const result = nums
  .filter(n => n > 2) // [3,4,5]
  .map(n => n * 10) // [30,40,50]
  .reduce((a, b) => a + b, 0); // 120
```

3. Modern JS Syntax

ES6+ features that make code cleaner, shorter, and easier to read.

Destructuring

```
// Array Destructuring
const [first, second, ...rest] = [10, 20, 30, 40];
// first=10, second=20, rest=[30,40]

// Object Destructuring
const { name, age, city = 'Unknown' } = { name: 'Alice', age: 30 };
// city defaults to 'Unknown' if missing

// Rename while destructuring
const { name: userName } = { name: 'Bob' };
// userName = 'Bob'

// In function parameters
function greet({ name, age }) {
  return `${name} is ${age}`;
}
```

Spread Operator (...)

```
// Spread arrays
const a = [1, 2, 3];
const b = [...a, 4, 5]; // [1,2,3,4,5]
const copy = [...a]; // shallow clone

// Spread objects
const base = { color: 'blue', size: 'md' };
const updated = { ...base, size: 'lg', weight: 'bold' };
// { color:'blue', size:'lg', weight:'bold' }
```

```
// Spread as function arguments
Math.max(...[3, 1, 4, 1, 5]); // 5
```

💡 *Spread creates a shallow copy — nested objects are still referenced, not deeply cloned.*

4. Architectural Patterns

Structuring JavaScript code for reuse, encapsulation, and maintainability.

ES6 Modules (import / export)

```
// math.js — named exports
export const PI = 3.14159;
export function add(a, b) { return a + b; }
export default class Calculator { /* ... */ }

// main.js — importing
import Calculator, { PI, add } from './math.js';
import * as MathUtils from './math.js'; // namespace import
```

Prototypal Inheritance

```
function Animal(name) {
  this.name = name;
}
Animal.prototype.speak = function() {
  return `${this.name} makes a sound.`;
};

function Dog(name) {
  Animal.call(this, name); // call parent constructor
}
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;
Dog.prototype.speak = function() {
  return `${this.name} barks.`;
};
```

ES6 Classes

Classes are syntactic sugar over prototypal inheritance — cleaner syntax, same prototype-based mechanism under the hood.

```
class Animal {
  #sound; // private field
  constructor(name, sound) {
    this.name = name;
    this.#sound = sound;
  }
  speak() { return `${this.name} says ${this.#sound}`; }
  static create(name) { return new Animal(name, '...'); }
}

class Dog extends Animal {
  constructor(name) { super(name, 'woof'); }
  fetch() { return `${this.name} fetches the ball!`; }
}

const d = new Dog('Rex');
```

```
d.speak(); // 'Rex says woof'
```

5. Asynchronous Engine

Understanding how JavaScript handles non-blocking operations despite being single-threaded.

Call Stack & Event Loop

```
// JavaScript is single-threaded — one call stack
// The EVENT LOOP picks tasks from the queue when stack is empty

console.log('1 - Start');           // Call Stack

setTimeout(() => {
  console.log('3 - Timeout');       // Macro-task Queue
}, 0);

Promise.resolve().then(() => {
  console.log('2.5 - Promise');     // Micro-task Queue (runs first!)
});

console.log('2 - End');             // Call Stack

// Output: 1 → 2 → 2.5 → 3
```

🔗 *Micro-tasks (Promises) always run before macro-tasks (setTimeout, setInterval) — even if the timeout is 0ms.*

Callbacks

The original pattern for async code. A function passed as an argument to be called once an operation completes.

```
function fetchData(url, onSuccess, onError) {
  setTimeout(() => {
    if (url) onSuccess({ data: 'result' });
    else onError(new Error('No URL'));
  }, 1000);
}

fetchData('/api/users',
  (data) => console.log(data),    // success callback
  (err)  => console.error(err)   // error callback
);

// ⚠️ Callback Hell: deeply nested callbacks become hard to manage
getUser(id, (user) => {
  getPosts(user.id, (posts) => {
    getComments(posts[0].id, (comments) => { /* ... */ });
  });
});
```

🔗 *Callback Hell is solved with Promises and async/await, which flatten the nesting into readable sequential code.*

6. Data Serialization

Converting JavaScript objects to and from JSON strings for API communication and state transfer.

JSON.stringify & JSON.parse

```
const user = { name: 'Alice', age: 30, active: true, tags: ['admin'] };

// Object → JSON string (for sending to API / localStorage)
const json = JSON.stringify(user);
// '{"name":"Alice","age":30,"active":true,"tags":["admin"]}'

// Pretty-print with indentation
const pretty = JSON.stringify(user, null, 2);

// JSON string → Object (received from API)
const parsed = JSON.parse(json);
parsed.name; // 'Alice'

// ⚠ JSON limitations
JSON.stringify({ fn: () => {}, sym: Symbol() }); // fn & sym are omitted!
JSON.stringify(undefined); // returns undefined
```

Working with APIs

```
async function getUsers() {
  try {
    const response = await fetch('https://api.example.com/users');

    if (!response.ok) throw new Error(`HTTP ${response.status}`);

    const data = await response.json(); // JSON.parse internally
    return data;
  } catch (err) {
    console.error('Fetch failed:', err);
  }
}

// POST with JSON body
await fetch('/api/users', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Bob', age: 25 }) // serialize
});
```

🔗 *`response.json()` is shorthand for `response.text().then(JSON.parse)`. Always check `response.ok` before parsing.*